RouterAcc: FPGA Acceleration for VLSI Detailed Router via Hierarchical Storage Mapping

ABSTRACT

The semiconductor industry keeps seeking for reducing the design time and efforts in modern integrated circuit implementation, while detailed routing is a dead-or-alive critical element in design automation tooling for advanced node enablement, which is known as the most time-consuming step in the back-end design flow due to its high complexity and enormous solution space for the very large-scale integration (VLSI) routing problem.

In this paper, we propose a FPGA-based implementation to accelerate the detailed routers, uses efficient memory access management and parallelism to achieve better performance. Experimental results demonstrate that detailed routers obtain stable and effective acceleration on FPGA. Our design gets 2.1×-2.3×, 2.1×-2.5×, 1.3× speedup compared to TritonRoute, Dr.CU,and current optimum routing kernel acceleration work, also obtains 20% speedup for detailed routing flow, and 1.0×-1.2× resource utilization compared to TritonRoute.

1 INTRODUCTION

Routing is known as the most complex and time-consuming stage in the physical design flow due to its high complexity and enormous solution space for the very large-scale integration (VLSI) routing problem. Routing is categorized into global routing and detailed routing. Global routing provides rough guidance with congestion and timing. Detailed routing generates exact rectilinear wiring interconnects based on global routing results and reduces the design rule violations to satisfy the manufacturability requirements. Detailed routing is the primary time bottleneck compared to global routing due to strict DRC adherence, larger search space, dynamic data management, manufacturing variability handling, and toolchain interoperability issues.

Recently detailed routers like TritonRoute[?] and Dr.CU 2.0[?] have demonstrated promising performance on industrial benchmarks from ISPD 2018/2019 contests. TritonRoute, as the CPU-driven routing engine within the state-of-the-art open-source RTL-to-GDSII flow, delivers design-rule-clean routing solutions for advanced technology nodes at 16nm and below. While this methodology demonstrates robust capability in resolving complex multi-constrained routing problems with high solution quality, its inherent computational inefficiency presents a significant bottleneck that markedly impacts turnaround time.

To accelerate routing, existing approaches primarily focus on utilizing partition-based multithreaded CPUs or GPU-accelerated architectures. However, these methods exhibit diminishing returns as thread counts increase beyond saturation points, often resulting in measurable quality degradation. By customizability of field-programmable gate array (FPGA), the realization of hardware and software co-acceleration with FPGA has already achieved competitive performance in accelerating computation tasks in fields like machine learning and scientific computing. Similarly, we can benefit from FPGA acceleration in the back-end algorithms by customizing and optimizing specific algorithms to provide higher performance and faster speed.

In this work, we propose MAccRouter, a parallelized iterative routing hardware framework integrated with memory access management enabled FPGA acceleration for detailed router. The key contributions are summarized as follows:

- A memory access management-enabled FPGA acceleration for detailed router is proposed.Instantiate multiple ram groups to store node information on the corresponding dimension, and Provides a flexible dimension partition architecture, RAM storage can be divided according to specific data characteristics according to X, Y, Z dimensions to maximize resource utilization.
- We propose efficient FPGA implementations of detailed routing Kernel on FPGA, and implement detailed routing methods based on grid and A* in workers.
- We demonstrate 2.1×-2.3× speedup in detailed routing without quality degradation of the entire flow over multi-threaded FPGA kernels implementations.More specifically,search operations are independent of each other so that the framework maintains nearly linear scalability with industrial designs.
- Through the trial of TritonRoute and Dr.CU accelerating, it is proved that the proposed optimization method has universal significance in the detailed routing scheme, and can be transferred to other similar work besides the example studied in this paper.

2 BACKGROUND

2.1 Detailed Routing Phases

During routing phases, a large number of design rules need to be processed. This process requires determining connection locations for all interconnects while satisfying design constraints at advanced technology nodes, fundamentally representing a path-planning problem with complex constraints in high-dimensional solution space, directly impacting chip performance, power consumption, and manufacturing costs. The extreme compression at nanometer scales and exponentially expanding circuit scale have made heuristic routing paradigms relying on iterative optimization particularly challenging, with the intensive computational demands on CPUs emerging as a critical bottleneck constraining both solution quality and optimization efficiency. To the escalating challenges posed by exponential growth in System-on-Chip (SoC) design complexity under post-Moore's Law paradigms, the open-source EDA flow "OpenROAD" has emerged as a cutting-edge solution for complete RTL-to-GDSII implementation. Across the design flow, running time is the critical metric and pressing task for IC design

automation, especially keeping peace with the quality of the routing resolution. Existing approaches primarily focus on utilizing partition-based multithreaded CPUs to accelerate routing.

In the Physical Design flow, routing is categorized into global routing and detailed routing due to the manufacturability limitations and large time consumption. Global routing provides rough guidance with congestion and timing. Detailed routing generates exact rectilinear wiring interconnects based on global routing results, and reduces the design rule violations to satisfy the manufacturability requirements. Unlike global routing, detailed routing infrastructure is built upon multiple tiers of metal layers. Within each layer, interconnects must be routed along predefined orthogonal tracks oriented either horizontally or vertically. Detailed routing is much more time-consuming than global routing due to several factors. First, detailed routing must comply with strict design rule constraints (DRC) and conduct multi-dimensional calculations, such as parasitic RC delay modeling and crosstalk analysis. Second, the path search space in detailed routing is much larger and more complex, as it operates on fine-grained grids and uses algorithms like maze routing to navigate obstacles. Third, detailed routing involves dynamic data management, with high-resolution layout databases requiring frequent DRC checks and incremental static timing analysis. Fourth, manufacturing variability necessitates statistical validation and redundancy mechanisms, consuming additional computational resources. Finally, toolchain interoperability issues, such as parsing DEF/LEF files and coordination with other modules, introduce significant delays. For example, a 7nm design might spend 48 hours on detailed routing compared to 3 hours for global routing.

In ISPD'18[1] and ISPD'19[?] detailed routing contest, detailed routing requires a combination of runtime, and constraints (e.g. minimum-cut, minimum-width, etc.) that have been formed into a better result. This necessitates a holistic consideration of tradeoffs between design rule adherence, manufacturability constraints, and multi-objective optimization metrics to balance routing quality with performance targets. Some excellent detailed routers like TritonRoute [5] and Dr.CU 2.0 [6] come to the fore in the competition. TritonRoute in particular, has been integrated into the OpenROAD open source process and has been adopted by several academic/industrial projects.

Prior research on detailed routing methodologies can be categorized into two complementary dimensions: (1) algorithmic advancements for conventional design rule-driven interconnect optimization, and (2) hardware-accelerated frameworks leveraging parallel computing architectures to overcome computational bottlenecks inherent in large-scale layout synthesis.

2.2 Memory Access Bottleneck

The core of routing algorithms lies in the highly efficient heuristic path-planning method, like the A-star (or A∗) search algorithm. However, the existing optimizations on the A* algorithm are mainly aimed at computational optimization, and the bottleneck in maintaining the open list and a closed list is obviously outstanding, especially when performing many iterative operations. For the typical path planning method (A*)acceleration by using FPGA, Zhou et al.[7] demonstrated 37-751× acceleration via 8-port cache and array-based OPEN lists, but the architecture suffers from inefficient resource utilization when processing high-obstacle density maps, where frequent cache misses degrade performance predictability.

Osama et al.[8] developed an FPGA implementation with 50-1001× speedup through 8-way parallel evaluators and cache-optimized memory access. Nevertheless, its fixed 256×256 grid mapping and complex 5×5 caching mechanism restrict scalability for large-scale scenarios. Jiang et al.[9] implemented an FPGA-based batched maze routing kernel achieving 3.11× speedup over CPU implementations with < 1% wirelength degradation, but its performance is constrained by data dependency management and limited scalability when handling heterogeneous net sizes. Meanwhile, Lin et al.[10] proposed GAMER, a GPU-accelerated maze routing scheme that achieved 161× acceleration for coarsened maze routing through sweeping-style parallelism. However, its performance gain diminishes in fine-grained routing stages, and the qualitydriven batch partitioning strategy introduces additional scheduling overhead that limits throughput.

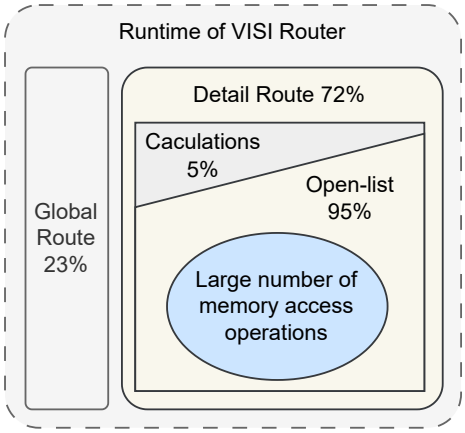


Figure 1: Detailed wiring takes up most of the time in the wiring process, and most of it is memory access time.

2.3 Versatility and Scalability

The above proposed acceleration methodology for detailed routing preserves the quality of the entire implementation flow for multithreaded FPGA core architectures. Actually, the accelerated A* techniques exhibit broad applicability across various path-searching implementations. Given the fundamental similarities in routing paradigms, particularly in maze routing algorithms and heuristic flow implementation methodologies, these acceleration strategies demonstrate significant transfer potential beyond the specific use cases examined in this study.

3 ALGORTHM

3.1 Detailed Routing Algorthm Analysis

Conventional framework of detailed routing algorithm we employ follows a structured flow, as illustrated in Algorithm 1. Given the global routing results of the benchmarks (e.g..def and.guides files),

we perform a track assignment step following the method described in [11], aligning all nets to predefined routing tracks. This preprocessing phase extracts and constructs essential data structures, such as routing grids, net objects, and guide constraints, which serve as inputs to the detailed routing engine. The ultimate goal is to produce a DRC-clean routing result with full connectivity, starting from these coarse global routing guides.

Algorithm 1 Detailed Routing Flow

Input: base-data structure, global routing guides
Output: full connected nets
1: InitDb(), InitPreRouteNet()
2: **while** $iter < maxIter$ **do**
3: $drcNets \leftarrow [net_i : while\ net_i \in Nets, if\ net_i.drcScore \neq 0]$
4: **while** $net \in drcNets$ **do**
5: $ucPins \leftarrow GetPins(net)$
6: **while** $ucPins \neq \emptyset$ **do**
7: $tpin \leftarrow \underset{pin \in ucPins}{\arg\max}\ pin.cost$
8: $cPins \leftarrow cPins \cup tPin, ucPins \leftarrow ucPins \setminus tPin$
9: $sPin \leftarrow$ GetNextPin($ucPins$)
10: FPGA-OptimizedRoutingKernel($sPin$,$cPins$)
11: **end while**
12: **end while**
13: UpdateDRC(), UpdateNets()
14: **end while**
15: UpdateDb()

The detailed routing flow, as depicted in Algorithm 1, comprises multiple sequential phases. Initially, the design database is initialized via InitDb(), and essential nets are pre-routed using InitPreRouteNet(). Following this, nets are evaluated based on their design rule check scores, and those exhibiting violations are prioritized for further processing. The outer loop (lines 2 to 14) performs iterative rip-up and reroute operations, bounded by a predefined maximum iteration count. In each iteration, the set of nets with non-zero DRC scores is extracted. For each net in drcNets, the set of unconnected pins is identified (line 5). The algorithm proceeds to incrementally connect these pins by repeatedly selecting the pin with the highest routing cost as the current target, moving it to the connected pin set, and identifying the next nearest source pin (lines 6–9). Routing between the selected source and target pins uses the Optimized Routing kernel, which leverages high-throughput hardware acceleration. This process continues until all pins in the net are connected. After net-level rerouting, the global DRC state and net data structures are updated accordingly . Upon convergence or exhaustion of iterations, a final database update is performed, reflecting the complete routing solution.

3.2 Routing Kernel and Generating Routing Graphs Algorithm Analysis

The most popular detailed routing methods, like TritonRoute[?] and Dr. CU[?], are both executed by the maze routing kernel to connect all pins on the nets with minimal cost. In the actual computing stage, the nets will first abstracted as grid graphs. Each pin pn is equivalent to a set of vertices Sn located on its position, then utilize the priority queue and some heuristic methods to improve the quality and speed.Taking TritonRoute[?] as an example, it uses A* (A-STAR) algorithm to explore the route and sets weight for each node. The weight is a heuristic function f (x) : f (x) = g(x) + h(x), where the function g(x) is the distance or cost from the starting node to current state x, and the function h(x) defines the estimated distance or cost from current state x to the end node. We call the function value of f (x) the f value.

Algorithm 2 FPGA-Optimized Routing Kernel Algorithm

Input: source pin s, target pins $\{t_1, \ldots, t_n\}$
Output: minimum-cost routing paths
Prepare: priority queue Q, cost map T, flag map F, graph batch $G\ \{G_1, ..., G_N\}$, direction set $D\ \{E, W, N, S, U, D\}$ in BRAM
1: Initialize N grid maps in BRAM
2: **for** $G_k \in G$ **in parallel do**
3: **set** $Q^{(k)}, T^{(k)}(v) \leftarrow \infty, F^{(k)}(v) \leftarrow 0, \forall v; T^{(k)}(s) \leftarrow 0$
4: **while** $Q^{(k)} \neq \emptyset$ **do**
5: $v \leftarrow$ MinCost($Q^{(k)}$)
6: $if\ v \in \{t_1, ..., t_n\}$ **break**
7: **for** $u \in D$ **in parallel do**
8: $c \leftarrow T^{(k)}(v) + \Delta(v, u)$
9: **if** $c < T^{(k)}(u)$ **then**
10: $T^{(k)}(u) \leftarrow c, F^{(k)}(u) \leftarrow 1, u.parent \leftarrow v$
11: Update($Q^{(k)}$)
12: **end if**
13: **end for**
14: **end while**
15: BackTrack Path
16: **end for**

In each round of A* search, we extract the state with the minimum f value from the open list, expand its outer neighbors and check for duplication. In general, Batched maze routing kernel is executed on a large batch of graphs in the detailed routing stage. There are almost 1,000 10,000 graphs (nets) in one batch without data dependency between any two graphs, providing a powerful condition for parallelization operation on FPGA to achieve efficient acceleration.

However, the typical maze routing algorithm should be modified to adapt the property of FPGA,as Algorithm 2 shows. The grid map is initialized in the memory and the start and end nodes are sent to the design so it can start solving the map. The priority queue Q, the temporary cost set T, and the vertices' state set F used in the hardware-optimized maze routing algorithm are also implemented on hardware. At the outer level, multiple routing graphs are processed concurrently, exploiting batch-level parallelism with thousands of nets in BRAM. At the inner level, for each graph, neighbour expansions across multiple directions are executed in parallel to accelerate wavefront propagation. This fine-grained directional parallelism, combined with hardware-managed cost updates and queue operations, enhances routing throughput on FPGA.

In the process of generating routing diagram, take TritonRoute[?] as an example, through the breadth-first search (BFS) path planning method of extending and updating wavefront, the wavefront is dynamically generated and explored in six directions, southeast,

northwest and upper, lower, and lower, to gradually update the cost information of each node to realize the path between the target point and the starting point. It can also be deployed on FPGAs for acceleration through full parallelism and data processing optimization.

4 DESIGN OF ACCELERATION MODULE

4.1 The Overview of FPGA Acceleration Framework

Conventional implementations of A* search usually use two lists to store the states during its expansion, i.e., the **open list** and the **closed list**. The closed list stores all the visited states, and is used to prevent unnecessary repeated expansion of the same state. This list is often implemented by a linked hash table to detect the duplicated nodes. The open list normally stores the states whose successors have not been fully explored yet. The open list uses a priority queue as its data structure, typically implemented by a binary heap. States in the open list are sorted according to the function value of heuristic function f (x).

A lot of work proves that memory access is the time bottleneck of the algorithm compared to the computation time, as Figure 1 shows. So our work is dedicated to propose a FPGA-based implementation to accelerate the detailed routers, uses efficient memory access management and parallelism to achieve better performance. Figure 2 shows the overview of FPGA acceleration framework.

4.2 Cost_Calculator Module

Cost_calculator module calculates the F-values of 6 peripheral points in parallel according to the input center point and surrounding point information. The information in 6 directions is calculated in parallel to the Manhattan distance, and the F value of the 6 surrounding points is calculated based on F=G+H.

4.3 Cost_Cache Module

Instantiate multiple ram groups to store node information on the corresponding dimension. Traditional FPGA-based path planning algorithms have significant bottlenecks in maintaining **open list** and **closed list**. The dynamic sorting mechanism requires the storage structure to be reordered after each search iteration. This frequent sorting operation will not only consume a lot of programmable logic resources, but also produce a processing delay that does not match the hardware architecture, which seriously restricts the real-time performance of the system. In addition, dynamic memory management of variable-length data structures leads to fragmented access to Block RAM, resulting in reduced hardware resource utilization.

To solve this problem we observed, the dimensional organization of RAM can be efficiently managed through hierarchical mapping, in FPGA-based storage architecture design, as fig1 shows. For 2D data structure, row-priority storage strategy can be adopted, so that each RAM module can store the data node corresponding to the row (first dimension) independently. When extended to a three-dimensional structure, each RAM module can be responsible for storing node information in a fixed plane, such as the third dimension. Through the introduction of address mapping module, high-dimensional addresses (such as three-dimensional coordinates (x,y,z)) can be converted into two-dimensional storage addresses inside each RAM module. This hierarchical mapping mechanism significantly improves storage access efficiency and data throughput.

4.4 F-value update Module

After the path cost calculation module completes the F-value generation of the adjacent nodes of the central node, the system executes the storage update process based on logical control. First, Cost_cache memory is accessed through the coordinate addressing mechanism to extract the status flag bit and historical F-value of the target node: If the flag bit is detected as the initial state (00), the new F-value and status flag are directly written without constraints; If the status flag is explored state (01), the hardware comparator is triggered to perform the numerical relationship determination of the new F-value and the stored F-value in parallel, and the path cost validity check is performed at the same time to avoid calculation anomalies. Finally, the storage unit is selectively updated according to the judgment result: for the initial state node to load new data completely, the explored state node only performs overwrite storage and synchronously updates the state when the new F-value is better. This mechanism uses the state-driven conditional write strategy to reduce the redundant access to memory and data overwrite operations while maintaining the principle of path planning optimality.

4.5 Minimum F-Value Sorting Module

In the process of selecting the least cost node of distributed storage architecture, the system uses the hierarchical comparison mechanism of self-updating to realize the dynamic update of the center point. The built-in hardware minimum value tracking module of each storage unit immediately triggers the parallel comparison circuit in the unit when a new F-value is detected: real-time comparison of the currently stored minimum F-value and the new written value through the special register group, dynamic update of the minimum cost value and its three-dimensional coordinate information, and generate an effective status flag bit to synchronize to the arbitration controller. When the valid flag bits of all storage units are ready, the hierarchical comparison architecture starts the global arbitration in stages. First, the local minimum values in each storage row are screened in parallel by row-level comparator array, and then the row-level minimum values are progressively compared by a tree arbiter based on depth-first strategy. Finally, the coordinates and path cost values of the globally optimal nodes are output.

After the selected global minimum is verified, the system loads the new central point information into the path planning logic through the three-dimensional coordinate bus. At the same time, double buffer register group is used to realize the decoupling of the computing unit and the storage unit to ensure the continuous operation of the comparison pipeline during the data update cycle. By means of distributed priority arbitration mechanism implemented by hardware, the architecture transforms the ergodic search in traditional algorithms into multilevel pipeline operation,

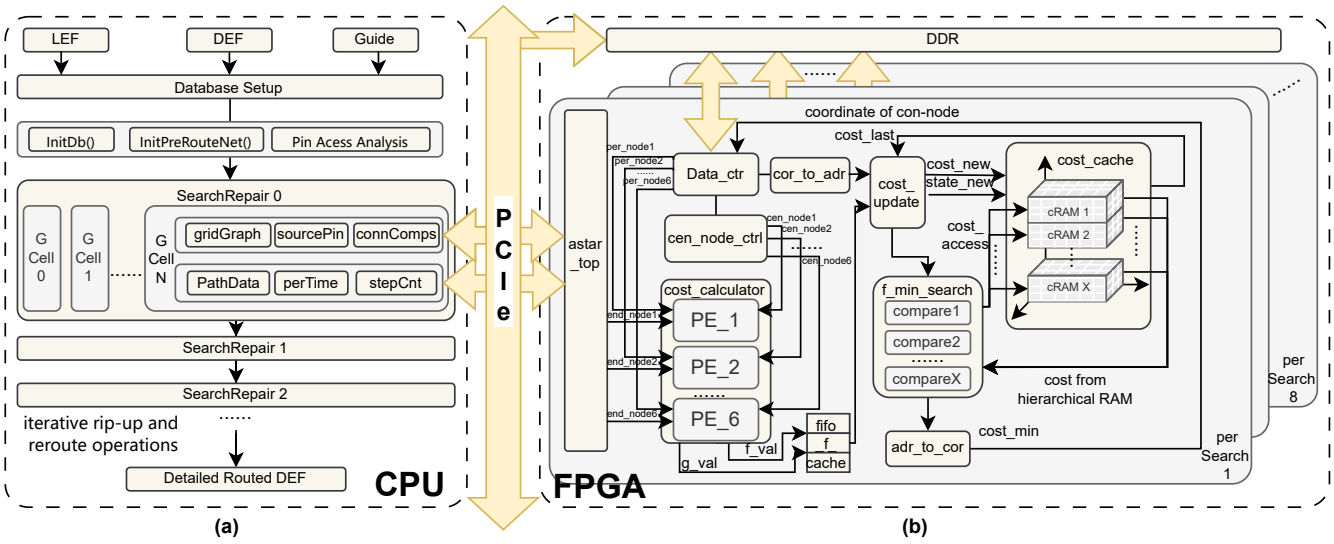


Figure 2: The overview structure.(a) The flow of detailed router on GPU.(b) The overview of FPGA acceleration framework.

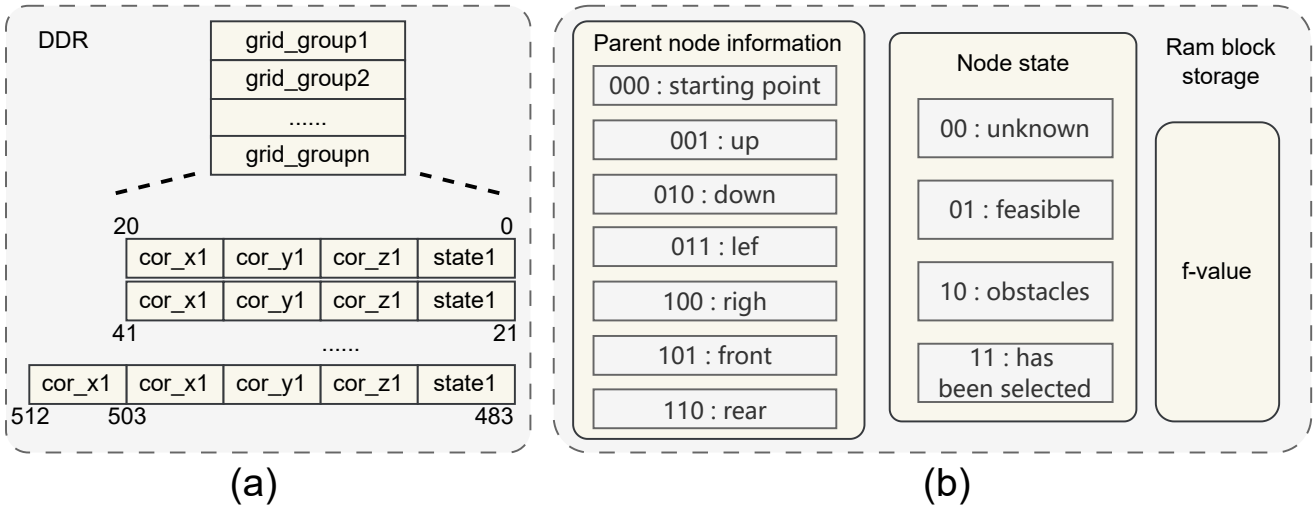


Figure 3: Data structure ON FPGA.(a)Data Layout in FPGA Memory DDR.(b)Storage format of multi-cRAM.

which avoids the resource competition problem of centralized comparison architecture while maintaining real-time path planning.

5 EXPERIMENTAL RESULTS

5.1 Experimental Setup

CPU:Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz,Memory:64G,8Thread;FPGA xcvu9p,8 computing cores in parallel

5.2 Maze Routing Acceleration

Table 1 shows the comparison of performance between TritonRoute [?] Dr.CU [?] and our FPGA acceleration to complete batched maze routing kernel with ISPD 2018 [?] contest benchmarks

5.3 Acceleration Efficiency Comparison

Table 2 shows comparation with other work.

5.4 Consumption of Hardware Resource.

Table 3 shows Consumption of Hardware Resource Utilization.

6 CONCLUSIONS

7 ACKNOWLEDGMENTS

REFERENCES

[1] Alan H Barr. 1991. The Einstein summation notation. *An Introduction to Physically Based Modeling (Course Notes 19), pages E* 1 (1991), 57.
[2] Ali Behrouz and Farnoosh Hashemi. 2024. Graph Mamba: Towards Learning on Graphs with State Space Models. *arXiv preprint arXiv:2402.08678* (2024).

Table 1: Comparison of performance between TritonRoute [?] Dr.CU [?] and our FPGA acceleration to complete batched maze routing kernel with ISPD 2018 [?] contest benchmarks.

Benchmark	ISPD' 18 quality score				Routing Kernel Runtime(s)				Routing Kernel Speed-up	
	TR	CU	MAcc-TR	MAcc-CU	TR	CU	MAcc-TR	MAcc-CU	MAcc-TR	MAcc-CU
ispd18_testl					4.41		2.03		2.2x	
ispd18_test2					58.86		25.93		2.3x	
ispd18_test3					110.07		52.4		2.1x	
ispd18_test4					244.48		211.68		2.3x	
ispd18_test5					109.42		40.39		2.3x	
ispd18_test6					165.39		77.33		2.1x	
ispd18_test7					246.12		111.98		2.2x	
ispd18_test8					256.73		125.29		2.1x	
ispd18_test9					264.21		128.19		2.1x	
ispd18_test10					380.69		180.455		2.1x	

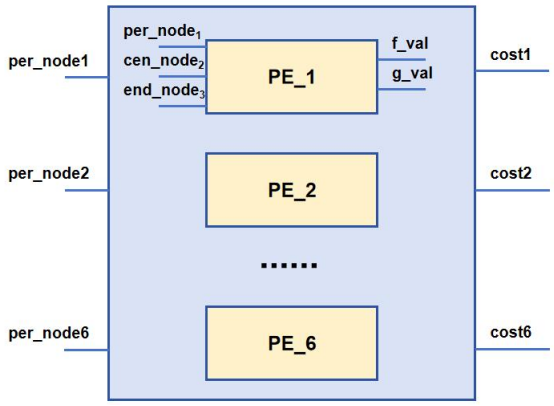


Figure 4: The Cost_Calculator Module.

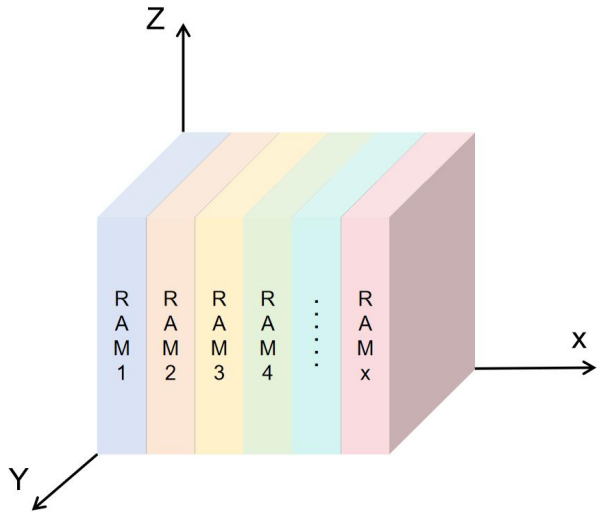


Figure 5: Instantiate multiple ram block storage

Table 2: Compare with other work[?].

ispd18	quality score		Routing Kernel (12 kernels)			
			Runtime(s)		Speed-up	
	[?] -CU	MAcc -CU	[?] -CU	MAcc -CU	[?] -CU	MAcc -CU
testl						
test2						
test3						
test4						
test5						
test6						
test7						
test8						
test9						
test10						

Table 3: Consumption of Hardware Resource Utilization.

Resource	Utilization	Available	Utilization %
LUT	48334	433200	11.16
LUTRAM	7760	174200	4.45
FF	49145	866400	5.67
BRAM	96.50	1470	6.56
IO	124	850	14.59
GT	8	36	22.22
BUFG	10	32	31.25
MMCM	4	20	20.00
PLL	1	20	5.00
PCIe	1	3	33.33

[3] Yonatan Bisk, Rowan Zellers, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *AAAI*.
[4] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
[5] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia a100 tensor core gpu: Performance and innovation.

IEEE Micro 41, 2 (2021), 29–35.
[6] Peter Clark, Isaac Cowhey, et al. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
[7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
[8] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks* 107 (2018), 3–11.
[9] Wade H Foy. 1976. Position-location solutions by Taylor-series estimation. *IEEE transactions on aerospace and electronic systems* 2 (1976), 187–194.
[10] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
[11] Albert Gu, Karan Goel, Ankit Gupta, and Christopher Ré. 2022. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems* 35 (2022), 35971–35983.
[12] Albert Gu, Karan Goel, and Christopher Ré. 2021. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396* (2021).
[13] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. 2021. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems* 34 (2021), 572–585.
[14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
[15] INTEL. 2024. Intel Xeon Platinum 8358P Processor. https://www.intel.com/content/www/us/en/products/sku/212308/intel-xeon-platinum-8358p-processor-48m-cache-2-60-ghz/specifications.html.
[16] Lincan Li, Hanchen Wang, Wenjie Zhang, and Adelle Coster. 2024. STG-Mamba: Spatial-Temporal Graph Learning via Selective State Space Model. *arXiv preprint arXiv:2403.12418* (2024).
[17] Xiuhong Li, Yun Liang, Shengen Yan, Liancheng Jia, and Yinghan Li. 2019. A coordinated tiling and batching framework for efficient GEMM on GPUs. In *Proceedings of the 24th symposium on principles and practice of parallel programming*. 229–241.
[18] Dingkang Liang, Xin Zhou, Xinyu Wang, Xingkui Zhu, Wei Xu, Zhikang Zou, Xiaoqing Ye, and Xiang Bai. 2024. PointMamba: A Simple State Space Model for Point Cloud Analysis. *arXiv preprint arXiv:2402.10739* (2024).
[19] Jun Liu, Guohao Dai, Hao Xia, Lidong Guo, Xiangsheng Shi, Jiaming Xu, Huazhong Yang, and Yu Wang. 2023. TSTC: Two-level Sparsity Tensor Core Enabling both Algorithm Flexibility and Hardware Efficiency. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
[20] Jiuming Liu, Ruiji Yu, Yian Wang, Yu Zheng, Tianchen Deng, Weicai Ye, and Hesheng Wang. 2024. Point mamba: A novel point cloud backbone based on state space model with octree-based ordering strategy. *arXiv preprint arXiv:2403.06467* (2024).
[21] Siyuan Lu, Meiqi Wang, Shuang Liang, Jun Lin, and Zhongfeng Wang. 2020. Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer. In *2020 IEEE 33rd International System-on-Chip Conference (SOCC)*. IEEE, 84–89.
[22] Haocong Luo, Yahya Can Tu, F Nisa Bostancı, Ataberk Olgun, A Giray Ya, Onur Mutlu, et al. 2023. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* (2023).
[23] Mishaim Malik, Muhammad Kamran Malik, Khawar Mehmood, and Imran Makhdoom. 2021. Automatic speech recognition: a survey. *Multimedia Tools and Applications* 80 (2021), 9411–9457.
[24] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S Vetter. 2018. Nvidia tensor core programmability, performance & precision. In *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, 522–531.
[25] Stephen Merity, Caiming Xiong, et al. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
[26] AI Meta. 2023. Introducing LLaMA: A foundational, 65-billion-parameter large language model. *Meta AI* (2023).
[27] Erxue Min, Runfa Chen, Yatao Bian, Tingyang Xu, Kangfei Zhao, Wenbing Huang, Peilin Zhao, Junzhou Huang, Sophia Ananiadou, and Yu Rong. 2022. Transformer for graphs: An overview from architecture perspective. *arXiv preprint arXiv:2202.08455* (2022).
[28] Naveen Muralimanohar, Rajeev Balasubramonian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to model large caches. *HP laboratories* 27 (2009), 28.
[29] Peter Nilsson, Ateeq Ur Rahman Shaik, Rakesh Gangarajaiah, and Erik Hertz. 2014. Hardware implementation of the exponential function using Taylor series. In *2014 NORCHIP*. IEEE, 1–4.
[30] NVIDIA. 2024. NVIDIA A100 Tensor Core GPU Architecture. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf.
[31] Mike O'Connor. 2014. Highlights of the high-bandwidth memory (hbm) standard. In *Memory forum workshop*, Vol. 3.
[32] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. The LAMBADA dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031* (2016).
[33] Johannes Partzsch, Sebastian Höppner, Matthias Eberlein, Rene Schüffny, Christian Mayr, David R Lester, and Steve Furber. 2017. A fixed point exponential function accelerator for a neuromorphic many-core system. In *2017 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–4.
[34] Badri N Patro and Vijay S Agneeswaran. 2024. SiMBA: Simplified Mamba-Based Architecture for Vision and Multivariate Time series. *arXiv preprint arXiv:2403.15360* (2024).
[35] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4195–4205.
[36] Enrico Reggiani, Renzo Andri, and Lukas Cavigelli. 2023. Flex-sfu: Accelerating dnn activation functions by non-uniform piecewise approximation. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
[37] Keisuke Sakaguchi, Ronan Le Bras, et al. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Commun. ACM* (2021).
[38] Nicol N Schraudolph. 1999. A fast, compact approximation of the exponential function. *Neural Computation* 11, 4 (1999), 853–862.
[39] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81.
[40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[41] Chloe Wang, Oleksii Tsepa, Jun Ma, and Bo Wang. 2024. Graph-mamba: Towards long-range graph sequence modeling with selective state spaces. *arXiv preprint arXiv:2402.00789* (2024).
[42] Teng Wang, Lei Gong, Chao Wang, Yang Yang, Yingxue Gao, Xuehai Zhou, and Huaping Chen. 2022. Via: A novel vision-transformer accelerator based on fpga. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4088–4099.
[43] Jan C Willems. 1986. From time series to linear system—Part I. Finite dimensional linear time invariant systems. *Automatica* 22, 5 (1986), 561–580.
[44] Yubiao Yue and Zhenzhang Li. 2024. Medmamba: Vision mamba for medical image classification. *arXiv preprint arXiv:2403.03849* (2024).
[45] Rowan Zellers, Ari Holtzman, et al. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).
[46] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
[47] Tao Zhang, Xiangtai Li, Haobo Yuan, Shunping Ji, and Shuicheng Yan. 2024. Point Could Mamba: Point Cloud Learning via State Space Model. *arXiv preprint arXiv:2403.00762* (2024).
[48] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. 2024. Vision mamba: Efficient visual representation learning with bidirectional state space model. *arXiv preprint arXiv:2401.09417* (2024).